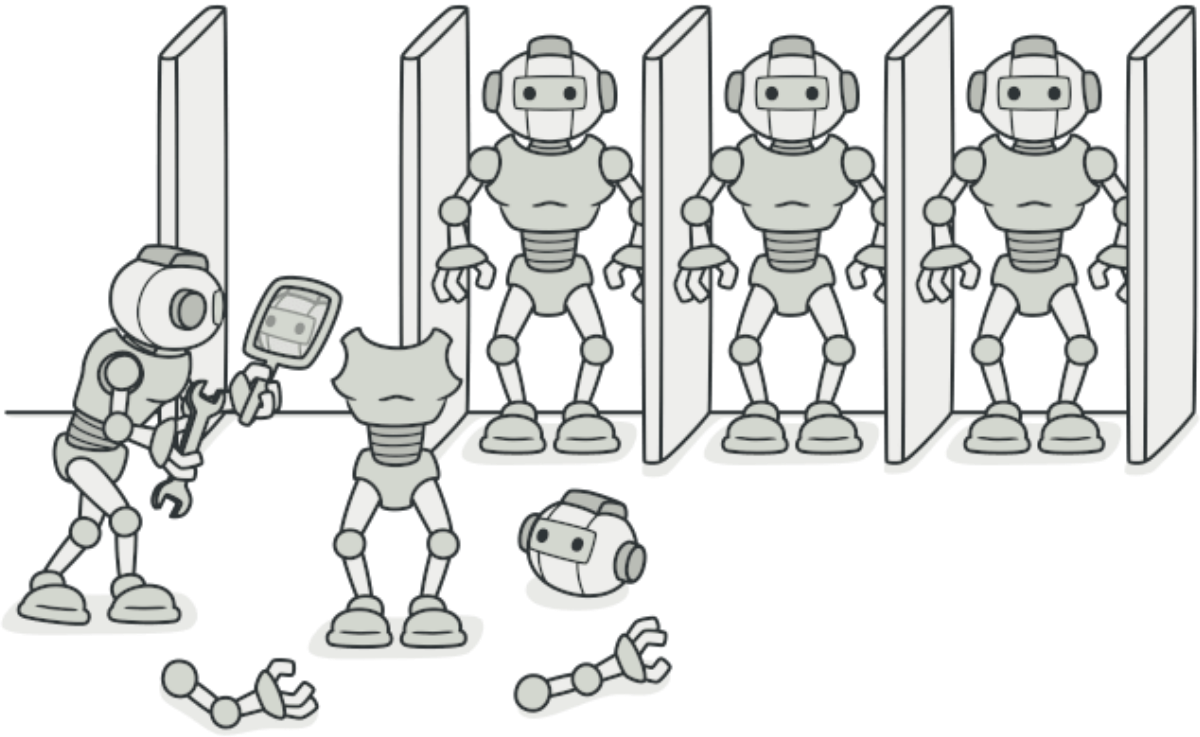
Prototype

Also known as: Clone

Absicht

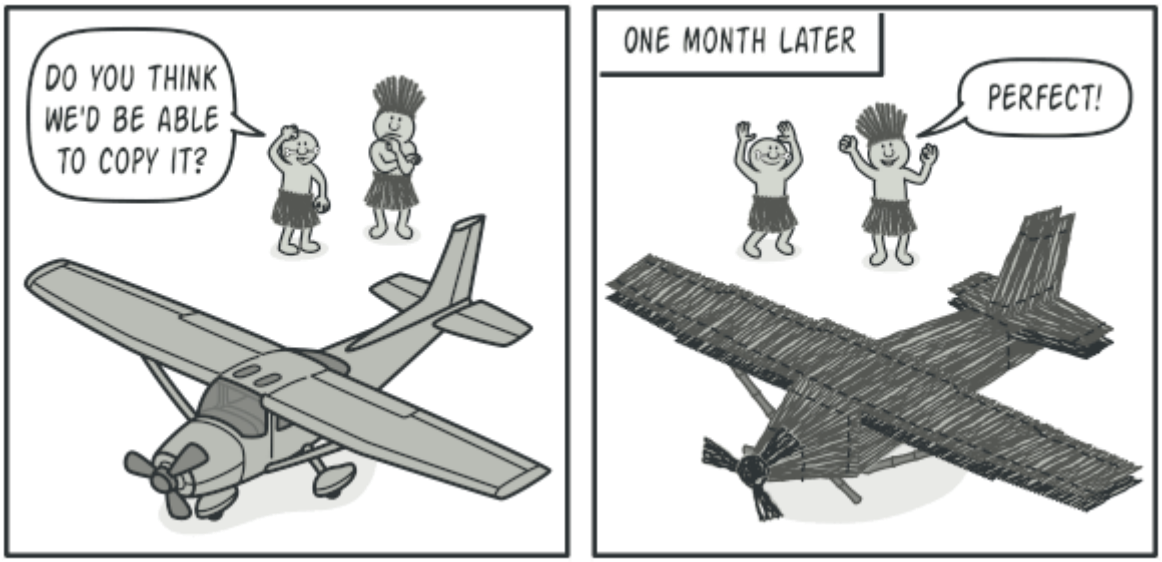
Prototype ist ein kreatives Entwurfsmuster, mit dem Sie vorhandene Objekte kopieren können, ohne Ihren Code von deren Klassen abhängig zu machen.



Problem

Angenommen, Sie haben ein Objekt und möchten eine exakte Kopie davon erstellen. Wie gehen Sie dabei vor? Zuerst müssen Sie ein neues Objekt derselben Klasse erstellen. Anschließend müssen Sie alle Felder des Originalobjekts durchgehen und deren Werte in das neue Objekt kopieren.

Schön! Aber es gibt einen Haken. Nicht alle Objekte können auf diese Weise kopiert werden, da einige Felder des Objekts privat sein können und von außerhalb des Objekts selbst nicht sichtbar sind.



Das Kopieren eines Objekts „von außen“ **ist nicht** immer möglich.
Der direkte Ansatz birgt noch ein weiteres Problem. Da Sie die Objektklasse kennen müssen, um ein Duplikat zu erstellen, wird Ihr Code von dieser Klasse abhängig. Auch wenn Sie diese zusätzliche Abhängigkeit nicht schreckt, gibt es einen weiteren Haken. Manchmal kennen Sie nur die Schnittstelle, der das Objekt folgt, nicht aber seine konkrete Klasse. Dies ist beispielsweise der Fall, wenn ein Parameter in einer Methode alle Objekte akzeptiert, die einer bestimmten Schnittstelle folgen.

Lösung

Das Prototype-Muster delegiert den Klonvorgang an die tatsächlich zu klonenden Objekte. Es deklariert eine gemeinsame Schnittstelle für alle Objekte, die das Klonen unterstützen. Diese Schnittstelle ermöglicht das Klonen eines Objekts, ohne den Code an die Klasse dieses Objekts zu koppeln. Normalerweise enthält eine solche Schnittstelle nur eine einzige `clone`Methode.

Die Implementierung der `clone`Methode ist in allen Klassen sehr ähnlich. Die Methode erstellt ein Objekt der aktuellen Klasse und überträgt alle Feldwerte des alten Objekts in das neue. Sie können sogar private Felder kopieren, da die meisten Programmiersprachen den Zugriff auf private Felder anderer Objekte derselben Klasse ermöglichen.

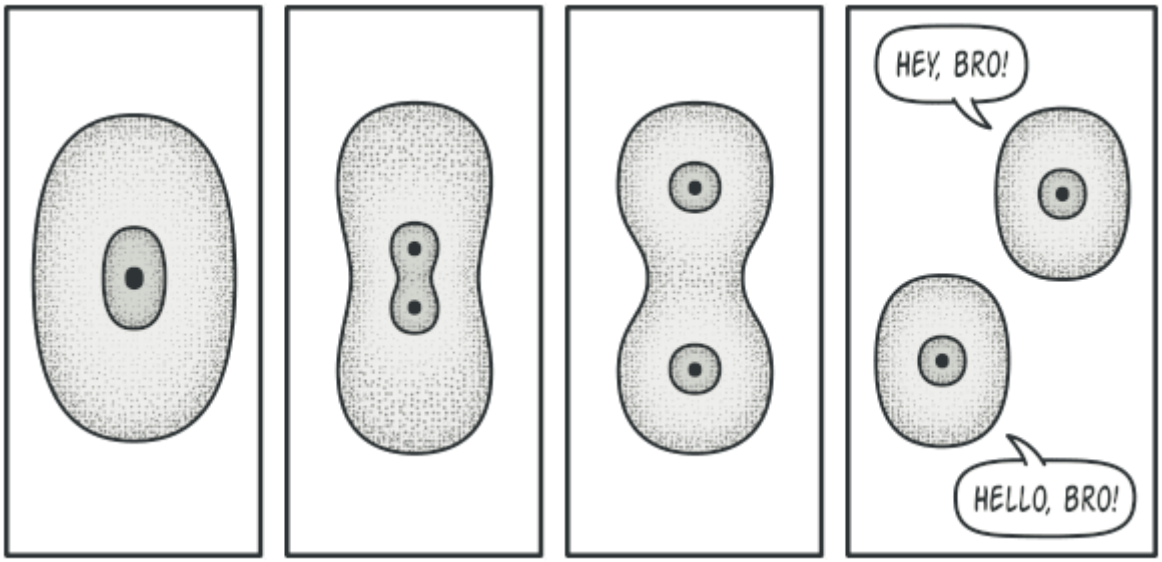
Ein Objekt, das das Klonen unterstützt, heißt *Prototyp* . Wenn Ihre Objekte Dutzende von Feldern und Hunderte von möglichen Konfigurationen haben, kann das Klonen eine Alternative zur Unterklassenbildung sein.



Vorgefertigte Prototypen können eine Alternative zur Unterklassenbildung sein. So funktioniert es: Sie erstellen eine Reihe von Objekten, die auf verschiedene Weise konfiguriert sind. Wenn Sie ein Objekt wie das von Ihnen konfigurierte benötigen, klonen Sie einfach einen Prototyp, anstatt ein neues Objekt von Grund auf neu zu erstellen.

Analogie zur realen Welt

In der Praxis werden Prototypen für verschiedene Tests verwendet, bevor mit der Massenproduktion eines Produkts begonnen wird. In diesem Fall nehmen Prototypen jedoch nicht an der eigentlichen Produktion teil, sondern spielen eine passive Rolle.



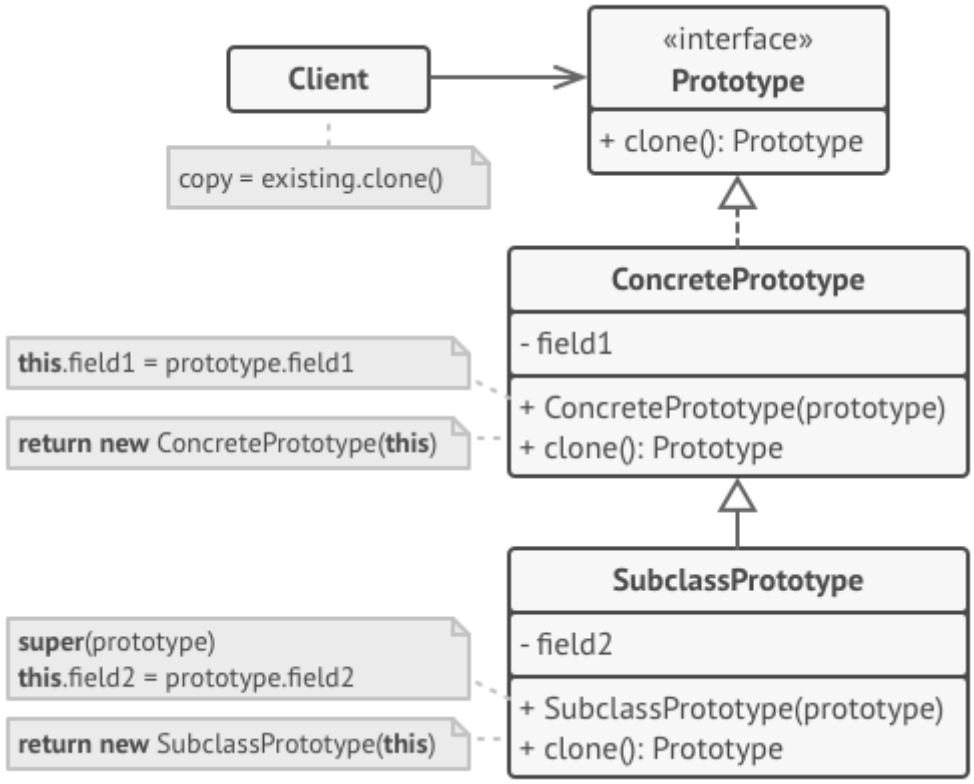
Die Teilung einer Zelle.

Da sich industrielle Prototypen nicht wirklich selbst kopieren, ist der Prozess der mitotischen Zellteilung (erinnern Sie sich noch an die Biologie?) eine viel genauere Analogie zum Muster. Nach der mitotischen Teilung bildet sich ein Paar identischer

Zellen. Die ursprüngliche Zelle fungiert als Prototyp und spielt eine aktive Rolle bei der Erstellung der Kopie.

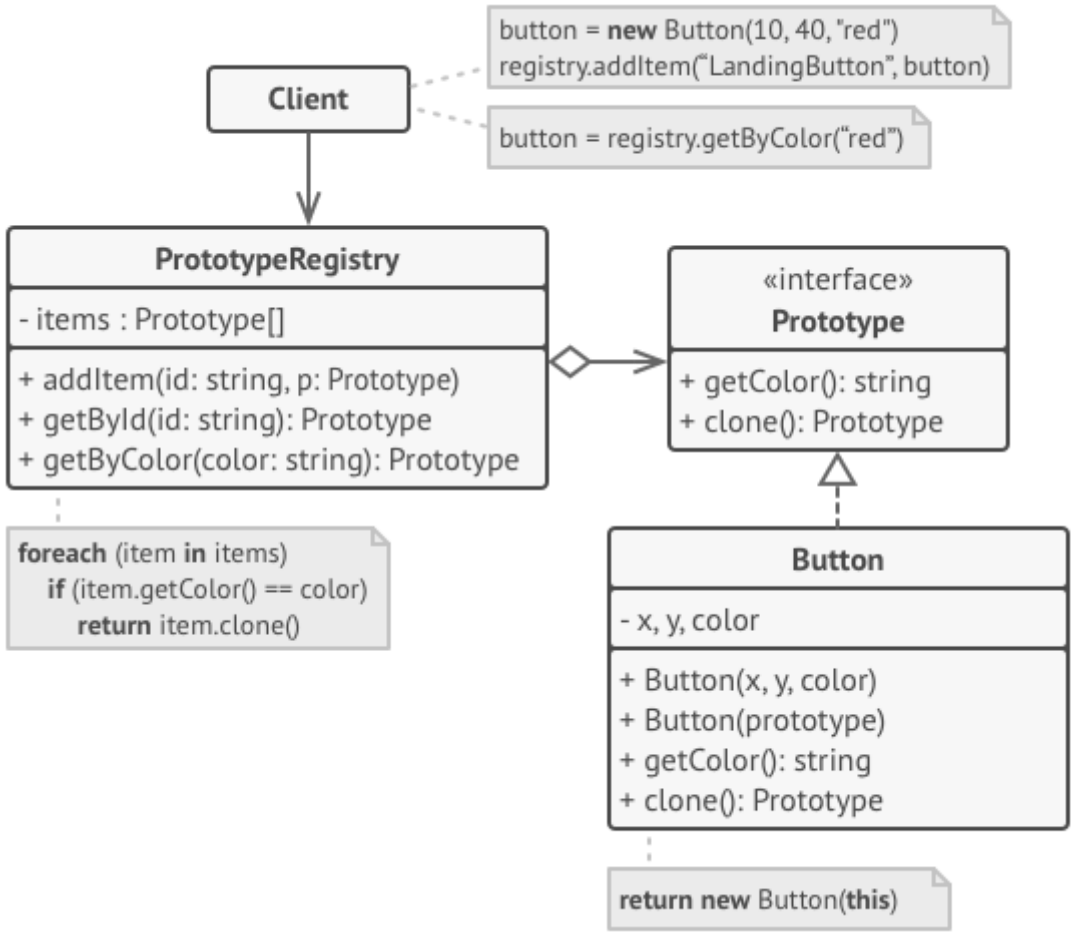
Struktur

Grundlegende Implementierung



1. Die **Prototype-** Schnittstelle deklariert die Klonmethoden. In den meisten Fällen handelt es sich um eine einzelne `clone`Methode.
2. Die Klasse **„Concrete Prototype“** implementiert die Klonmethode. Neben dem Kopieren der Daten des Originalobjekts in den Klon kann diese Methode auch Randfälle des Klonprozesses behandeln, z. B. das Klonen verknüpfter Objekte, das Auflösen rekursiver Abhängigkeiten usw.
3. Der **Client** kann eine Kopie jedes Objekts erstellen, das der Prototypschnittstelle folgt.

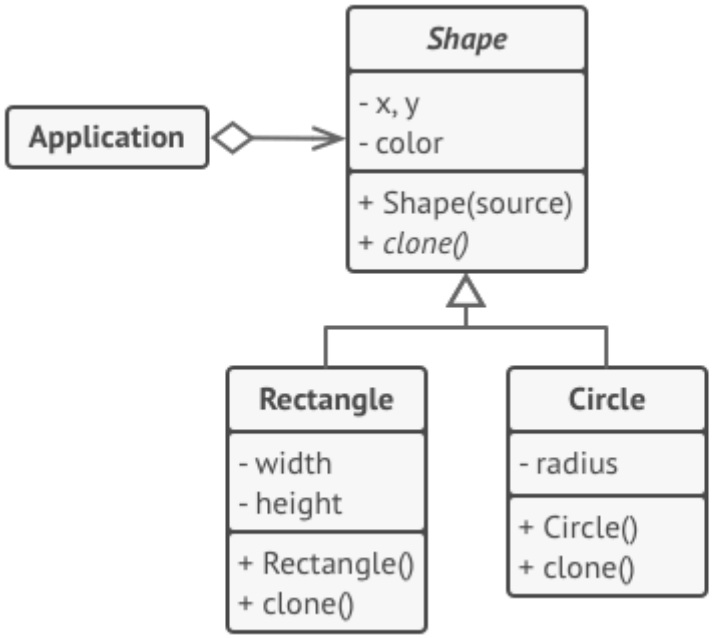
Prototypische Registrierungsimplementierung



1. Die **Prototype Registry** bietet einen einfachen Zugriff auf häufig verwendete Prototypen. Sie speichert eine Reihe vorgefertigter Objekte, die zum Kopieren bereitstehen. Die einfachste Prototype Registry ist eine `name → prototype`Hash-Map. Wenn Sie jedoch bessere Suchkriterien als einen einfachen Namen benötigen, können Sie eine deutlich robustere Version der Registry erstellen.

Pseudocode

In diesem Beispiel können Sie mit dem **Prototypmuster** exakte Kopien geometrischer Objekte erstellen, ohne den Code an ihre Klassen zu koppeln.



Klonen einer Gruppe von Objekten, die zu einer Klassenhierarchie gehören. Alle Shape-Klassen folgen derselben Schnittstelle, die eine Klonmethode bereitstellt. Eine Unterklasse kann die Klonmethode der übergeordneten Klasse aufrufen, bevor sie ihre eigenen Feldwerte in das resultierende Objekt kopiert.

```
// Base prototype.
abstract class Shape is
    field X: int
    field Y: int
    field color: string

    // A regular constructor.
    constructor Shape() is
        // ...

    // The prototype constructor. A fresh object is initialized
    // with values from the existing object.
    constructor Shape(source: Shape) is
        this()
        this.X = source.X
        this.Y = source.Y
        this.color = source.color

    // The clone operation returns one of the Shape subclasses.
    abstract method clone():Shape


// Concrete prototype. The cloning method creates a new object
// in one go by calling the constructor of the current class and
// passing the current object as the constructor's argument.
// Performing all the actual copying in the constructor helps to
// keep the result consistent: the constructor will not return a
// result until the new object is fully built; thus, no object
// can have a reference to a partially-built clone.
```

```
class Rectangle extends Shape is
    field width: int
    field height: int

    constructor Rectangle(source: Rectangle) is
        // A parent constructor call is needed to copy private
        // fields defined in the parent class.
        super(source)
        this.width = source.width
        this.height = source.height

    method clone():Shape is
        return new Rectangle(this)


class Circle extends Shape is
    field radius: int

    constructor Circle(source: Circle) is
        super(source)
        this.radius = source.radius

    method clone():Shape is
        return new Circle(this)


// Somewhere in the client code.
class Application is
    field shapes: array of Shape

    constructor Application() is
        Circle circle = new Circle()
        circle.X = 10
        circle.Y = 10
        circle.radius = 20
        shapes.add(circle)

        Circle anotherCircle = circle.clone()
        shapes.add(anotherCircle)
        // The `anotherCircle` variable contains an exact copy
        // of the `circle` object.

        Rectangle rectangle = new Rectangle()
        rectangle.width = 10
        rectangle.height = 20
        shapes.add(rectangle)

    method businessLogic() is
```

```
// Prototype rocks because it lets you produce a copy of
       // an object without knowing anything about its type.
       Array shapesCopy = new Array of Shapes.

       // For instance, we don't know the exact elements in the
       // shapes array. All we know is that they are all
       // shapes. But thanks to polymorphism, when we call the
       // `clone` method on a shape the program checks its real
       // class and runs the appropriate clone method defined
       // in that class. That's why we get proper clones
       // instead of a set of simple Shape objects.
       foreach (s in shapes) do
           shapesCopy.add(s.clone())

       // The `shapesCopy` array contains exact copies of the
       // `shape` array's children.
```

Anwendbarkeit

Verwenden Sie das Prototypmuster, wenn Ihr Code nicht von den konkreten Objektklassen abhängen soll, die Sie kopieren müssen.
Dies passiert häufig, wenn Ihr Code mit Objekten arbeitet, die Ihnen über eine Schnittstelle von Drittanbietercode übergeben werden. Die konkreten Klassen dieser Objekte sind unbekannt, und Sie könnten sich nicht auf sie verlassen, selbst wenn Sie wollten.

Das Prototype-Muster stellt dem Clientcode eine allgemeine Schnittstelle für die Arbeit mit allen Objekten bereit, die das Klonen unterstützen. Diese Schnittstelle macht den Clientcode unabhängig von den konkreten Objektklassen, die er klont.

Verwenden Sie das Muster, wenn Sie die Anzahl der Unterklassen reduzieren möchten, die sich nur in der Art und Weise unterscheiden, wie sie ihre jeweiligen Objekte initialisieren.
Angenommen, Sie haben eine komplexe Klasse, die vor ihrer Verwendung eine aufwändige Konfiguration erfordert. Es gibt mehrere gängige Möglichkeiten, diese Klasse zu konfigurieren, und dieser Code ist über Ihre App verteilt. Um die Duplizierung zu reduzieren, erstellen Sie mehrere Unterklassen und fügen den gesamten gängigen Konfigurationscode in deren Konstruktoren ein. Sie haben das Duplizierungsproblem gelöst, verfügen nun aber über zahlreiche Dummy-Unterklassen.

Mit dem Prototype-Muster können Sie eine Reihe vorgefertigter Objekte, die auf verschiedene Weise konfiguriert sind, als Prototypen verwenden. Anstatt eine Unterklasse zu instanziieren, die einer bestimmten Konfiguration entspricht, kann der Client einfach nach einem geeigneten Prototyp suchen und diesen klonen.

So implementieren Sie

1. Erstellen Sie die Prototypschnittstelle und deklarieren Sie die `clone`Methode darin. Oder fügen Sie die Methode einfach allen Klassen einer vorhandenen Klassenhierarchie hinzu, falls vorhanden.
2. Eine Prototypklasse muss den alternativen Konstruktor definieren, der ein Objekt dieser Klasse als Argument akzeptiert. Der Konstruktor muss die Werte aller in der Klasse definierten Felder aus dem übergebenen Objekt in die neu erstellte Instanz kopieren. Wenn Sie eine Unterklasse ändern, müssen Sie den übergeordneten Konstruktor aufrufen, damit die Oberklasse das Klonen ihrer privaten Felder übernimmt.

   Wenn Ihre Programmiersprache keine Methodenüberladung unterstützt, können Sie keinen separaten „Prototyp“-Konstruktor erstellen. Das Kopieren der Objektdaten in den neu erstellten Klon muss daher innerhalb der `clone`Methode erfolgen. Dennoch ist es sicherer, diesen Code in einem regulären Konstruktor zu speichern, da das resultierende Objekt direkt nach dem Aufruf des `new`Operators vollständig konfiguriert zurückgegeben wird.

3. Die Klonmethode besteht üblicherweise aus nur einer Zeile: dem Ausführen eines `new`Operators mit der prototypischen Version des Konstruktors. Beachten Sie, dass jede Klasse die Klonmethode explizit überschreiben und ihren eigenen Klassennamen zusammen mit dem `new`Operator verwenden muss. Andernfalls kann die Klonmethode ein Objekt einer übergeordneten Klasse erzeugen.
4. Erstellen Sie optional ein zentrales Prototypenregister, um einen Katalog häufig verwendeter Prototypen zu speichern.

   Sie können die Registrierung als neue Factory-Klasse implementieren oder sie mit einer statischen Methode zum Abrufen des Prototyps in die Basisprototypklasse einfügen. Diese Methode sucht anhand der Suchkriterien, die der Clientcode an die Methode übergibt, nach einem Prototyp. Die Kriterien können entweder ein einfacher String-Tag oder ein komplexer Satz von Suchparametern sein. Nachdem der passende Prototyp gefunden wurde, klont die Registrierung ihn und gibt die Kopie an den Client zurück.

   Ersetzen Sie abschließend die direkten Aufrufe der Konstruktoren der Unterklassen durch Aufrufe der Factory-Methode des Prototyp-Registers.

Für und Wider

- Sie können Objekte klonen, ohne sie an ihre konkreten Klassen zu koppeln.
- Sie können wiederholten Initialisierungscode loswerden und stattdessen vorgefertigte Prototypen klonen.
- Sie können komplexe Objekte bequemer herstellen.
- Sie erhalten eine Alternative zur Vererbung beim Umgang mit Konfigurationsvorgaben für komplexe Objekte.

- Das Klonen komplexer Objekte mit zirkulären Referenzen kann sehr schwierig sein.

Beziehungen zu anderen Mustern

- Viele Designs beginnen mit der Verwendung **der Factory-Methode** (weniger kompliziert und über Unterklassen besser anpassbar) und entwickeln sich in Richtung **Abstract Factory** , **Prototype** oder **Builder** (flexibler, aber komplizierter).
- **Abstrakte Factory** -Klassen basieren häufig auf einer Reihe von **Factory-Methoden** , Sie können jedoch auch **Prototype** verwenden , um die Methoden dieser Klassen zusammenzustellen.
- **Prototype kann hilfreich sein, wenn Sie Kopien von Befehlen** im Verlauf speichern müssen .
- **Designs, die häufig Composite** und **Decorator** verwenden , profitieren oft von der Verwendung von **Prototype** . Durch die Anwendung des Musters können Sie komplexe Strukturen klonen, anstatt sie von Grund auf neu zu erstellen.
- **Prototype** basiert nicht auf Vererbung und hat daher keine Nachteile. Andererseits erfordert *Prototype* eine komplizierte Initialisierung des geklonten Objekts. **Factory Method** basiert auf Vererbung, erfordert aber keinen Initialisierungsschritt.
- Manchmal ist **Prototype eine einfachere Alternative zu Memento** . Dies funktioniert, wenn das Objekt, dessen Zustand Sie im Verlauf speichern möchten, relativ einfach ist und keine Links zu externen Ressourcen enthält oder die Links leicht wiederhergestellt werden können.
- **Abstrakte Fabriken** , **Builder** und **Prototypen** können alle als **Singletons** implementiert werden .